Vasic-Digital-Reusable-Module-Suite

Build once, reuse everywhere — a fleet of small, decoupled, independently-tested Go and KMP modules.

Summary

A large family of generic, reusable modules published under the `digital.vasic.*` (Go) and Kotlin Multiplatform namespaces. Every module is standalone, independently tested and versioned, and consumed as an equal-codebase submodule by larger products (Catalogizer, HelixAgent, and the wider fleet). This page consolidates the many small utilities that would be noise as individual pages.

Short description

A curated suite of decoupled `digital.vasic.*` modules — infrastructure primitives (auth, cache, database, config, observability), AI/agent building blocks (RAG, VectorDB, Embeddings, MCP, Agentic, Planning), and defensive-LLM guardrails (RedTeam, Normalize) — plus a Kotlin Multiplatform mirror set. Each is generic, tested, and reusable.

Long description

The vasic-digital org runs on one structural bet: a “constitution + many decoupled reusable submodules” philosophy in which generic functionality is never written twice. Instead of monoliths, every reusable concern is extracted into its own small module — its own repository, its own tests, its own docs — and held strictly decoupled so no consumer’s specifics ever leak in. This page groups them because, taken one at a time, each is library-scale and would be noise as an individual product page. Taken together, they are the org’s real force multiplier: a private engineering asset that turns “build a new product” into “assemble proven

parts," and the concrete backing for the claim that this fleet does not reinvent the wheel — it maintains one very good wheel and rolls it everywhere.

The suite spans three clusters. **Infrastructure primitives** (Go) provide the plumbing every service needs: `auth` (JWT/bcrypt), `cache` (Redis/TTL), `database` (migrations, dual SQLite/PostgreSQL), `config`, `middleware`, `observability` (Prometheus/OpenTelemetry), `ratelimiter`, `security`, `storage` (S3/MinIO), `streaming` (WebSocket hub), `eventbus`, `filesystem` (multi-protocol), `discovery/mdns`, `http3`, `recovery`, `concurrency`, `lazy`, and more. **AI/agent building blocks** (Go) provide the substrate for AI systems: `rag`, `vectordb`, `embeddings`, `memory`, `conversation` (infinite-context compression, event sourcing), `mcp` (Model Context Protocol), `toolschema`, `skillregistry`, `agentic` (graph-based workflow orchestration), `planning` (HiPlan/MCTS/Tree-of-Thoughts), `benchmark` (SWE-bench/HumanEval/MMLU), `llmops`, `selfimprove` (reward modeling/RLHF), and `toon` (Token-Oriented Object Notation). **Defensive-LLM guardrails** provide adversarial-robustness tooling: `RedTeam` (YAML-driven adversarial fixtures), `Normalize` (adversarial-input canonicalisation). A parallel **Kotlin Multiplatform** set mirrors core modules (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP, etc.) for cross-platform apps.

Why we built it

Shipping many products (Catalogizer, HelixAgent, Herald, and more) from scratch each time is wasteful and inconsistent. Extracting every generic concern into a decoupled, tested module means fixes and improvements propagate across the whole fleet, and each new product assembles from proven parts.

Why it's a game-changer

It is, in effect, a private "standard library" for building AI-centric backends — the layer most teams never get to build because they are too busy re-solving auth, caching, and RAG plumbing for the fifth time. Here infrastructure primitives, AI building blocks, and defensive-LLM guardrails all exist as drop-in, independently-tested modules, which is what lets a small team ship product-grade systems at a pace that normally requires a much larger one, and do it without the duplication debt that usually accrues in its wake.

What's innovative

- Fleet-wide decoupling discipline (CONST-051): submodules treated as equal codebases, never carrying consumer specifics.

- A dedicated AI-primitive layer (RAG, VectorDB, Embeddings, MCP, ToolSchema, Agentic, Planning, LLMOps) as reusable modules.
- A defensive-LLM guardrail cluster (RedTeam, Normalize) for adversarial robustness.
- Parallel Go + Kotlin Multiplatform module sets sharing the same conventions.

Challenges & solutions

- **Avoiding coupling rot across dozens of modules:** solved with the constitution's decoupling contract and runtime injection of consumer specifics.
- **Keeping many modules consistent and tested:** solved with a shared convention (per-module tests/docs/Challenges) and the HelixConstitution governance backbone.
- **Cross-platform reach:** solved with a Kotlin Multiplatform mirror of core modules.

Tech stack (why + how)

- **Go** — the majority of modules (`digital.vasic.*`).
- **Kotlin Multiplatform** — cross-platform mirror modules (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** — cache, database, storage primitives.
- **Prometheus / OpenTelemetry** — observability module.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — networking modules.
- **Vector DB / embeddings / RAG / MCP** — AI-primitive modules.
- **YAML** — RedTeam adversarial fixtures and config.

> UNVERIFIED / WIP: several org repos are self-marked "SCAFFOLD / WIP" (e.g. `PliniusCommon`, `I-LLM`, `HyperTune`, `AutoTemp`, `Veritas`, `Ouroborous`, `Claritas`, `LeakHub`, `GandalfSolutions`). Present these as early-stage/scaffold, not shipped.